Data Structures Visualizer

Course: CS163 – Data Structures

Châu Tấn Phát

Group ID: 55A

April 28, 2026

Contents

1. Abstract **3**

2. Introduction **3**

3. Data Storage **3**

4. Project Architecture **3**

5. Implementation Details **4**

5.1. Core structures 4

5.2. Main functions 5

5.2.1. Algorithmic Logic 5

5.2.2. Positioning & Layout 5

5.2.3. Animation Controls 5

5.2.4. History & Snapshot Management 5

5.3. Main flow example: AVL Tree Insertion 5

6. Technical Problems and Solutions **6**

6.1. Backward logic 6

6.2. Graph node overlap 6

6.3. Compiler conflicts with Windows GUI 6

7. Features Demonstration **6**

8. Conclusion **6**

9. References **6**

Data Structures Visualizer

1. Abstract

The **DSA Visualizer** is an interactive, desktop-based graphical application designed to help students understand complex Data Structures and Algorithms (DSA). Built entirely in C++ using the raylib framework and raygui, the project visualizes the step-by-step animation of fundamental and advanced data structures, including Linked Lists, Binary Heaps, AVL Trees, AA Trees, Minimum Spanning Trees (Kruskal), and Shortest Path (Dijkstra). It features pseudocode highlighting, interactive graph rendering, and a snapshot system, allowing users to go to the next step, go to the previous step, and go to the final step.

2. Introduction

Data structures and algorithms are one of the fundamental concepts of computer science, yet many learners struggle with abstract concepts like tree rotations, heapifications, and graph traversals. Traditional learning methods often rely on static textbook images or dry terminal outputs.

This project was created to bridge the gap between theoretical algorithmic logic and visual comprehension. The primary objective is to provide a smooth, interactive sandbox where users can insert, delete, update, and search for nodes while watching the visualization of underlying steps. By combining these animations with a synchronized pseudocode panel, the user can trace exactly which line of code produces which step in the visualization process.

3. Data Storage

Data in this project is strictly managed in-memory to ensure maximum performance and responsiveness during rendering.

- Structures are stored using actual C++ representations of the algorithms (node pointers for `LinkedList`, `AVL`, and `AA` trees; `std::vector` for arrays in `Heap` and edge lists in graphs).
- State Tracking (Snapshots): Instead of implementing reverse algorithms, the visualizer stores the state of each step. Before taking a new step, a `Snapshot` struct is captured and pushed into a `std::vector<Snapshot> history`. This deeply copies the visual states, variable pointers, and active lines of pseudocode, allowing $O(1)$ state restoration when the user clicks the "Backward" or "Start" buttons.

4. Project Architecture

The overall architecture of the project can be viewed as this directory tree:

```
dsa-visualizer/
├── src/
├── header/
├── vendor/
├── inputs/
├── fonts/
```

The project follows a modular, C++–driven structure:

- `src/` (Source Code): Contains the core algorithmic logic and rendering implementations (`linkedlist.cpp`, `avl.cpp`, `shortest_path.cpp`, etc.). `main.cpp` handles the global application loop and routing, while `common.cpp` stores common functions that can be used by all struc-

tures, like `DrawCustomButton` and `DrawCodePanel`, and `menu.cpp` handles the drawing of the menu of the visualizer.

- `header/` (Declarations): Contains the `.h` interfaces for all data structures to keep dependencies clean and compilation fast.
- `vendor/` (Third-Party Libraries): Houses lightweight, single-file libraries such as `raygui.h` for the graphical user interface and `tinyfiledialogs.h` for file uploading capabilities.
- `inputs/` (Sample inputs): Some sample inputs, text files for testing the data structures, using the file uploading feature.
- `fonts/` (Sample inputs): Contains the fonts of the application.

Here are some of the data structures and algorithms that I have implemented for the project:

- **Linked List**: While a `vector` offers $O(1)$ random access, a Linked List is chosen for scenarios requiring frequent insertions and deletions at arbitrary positions without shifting elements.
- **Binary Heap (Max Heap):** An unsorted array requires $O(n)$ to find the minimum, while a sorted list requires $O(n)$ for insertion. The Heap provides a balanced trade-off with $O(\log n)$ for both insertion and extraction of the minimum element, which is essential for scaling graph algorithms.
- **AVL Tree:** Unlike a standard BST, which has the complexity $O(n)$ in the worst case, the AVL Tree maintains a height of $O(\log n)$ by enforcing a balance factor. It is chosen over other data strucutre when because the operations in AVL Tree are much easier to implement.
- **AA Tree**: Standard Red-Black Trees have many edge cases for rotation and recoloring, therefore require heavy implementation. The AA Tree simplifies this by only allowing right-level links (eliminating many "left-leaning" cases), making the implementation significantly more maintainable while maintaining the same $O(\log n)$ performance guarantees. The reason I chose this data structure is also that this data structure has a lot in common with the AVL Tree. Therefore, I can reuse many implemented functions of the AVL Tree for the AA Tree.

5. Implementation Details

5.1. Core structures

- `struct LinkedList`, `AVL`, `AA`, `Heap`: Each structure holds an internal `Node` struct containing positional coordinates (x, y, vX, vY) alongside standard algorithmic variables (`value`, `left`, `right`, `height`).
- `struct DIJKSTRA`, `MST`: Relies on a `Node` vector for vertices and an `Edge` struct for connections, applying union-find sets or distance arrays for logic.
- Node: The fundamental building block of the data structure.
  - `value`: The primary data stored in the node.
  - `level / height / balance`: Metadata required for self-balancing algorithms.
  - `id`: A unique identifier used to track nodes across different snapshots.
  - `x, y`: Coordinates of the nodes.
  - `left, right, parent`: Pointers defining the hierarchical relationships.
- `struct Snapshot`: Embedded inside each class. It captures the exact frame data, including the `activeLine` for the code panel and `animMode` tracking variables.
  - `NodeState`: A lightweight version of a Node containing only essential data (`id, value, position`) for reconstruction.

- `states`: A list of all `NodeState` objects currently in the structure.
- `activeLine`: The specific line of pseudocode being executed during this snapshot.

5.2. Main functions

The methods within each data structure class are organized by their responsibility, and can be divided into the following categories:

5.2.1. Algorithmic Logic

Functions responsible for the main logics of the data structures:

- `insertLogic() / deleteLogic()`: Recursive or iterative implementations of the core algorithm.
- `skew() / split() / rotate()`: Balancing operations specific to trees (e.g., AVL or AA).
- `search()`: Traversal logic to locate specific values.

5.2.2. Positioning & Layout

Functions that translate logical relationships into 2D space:

- `calculatePositions()`: A recursive function that assigns x and y coordinates to nodes based on their depth and sibling count to prevent overlapping.
- `drawLinkedList / drawTree() / drawGraph`: Used to render nodes and edges to the screen.

The visualizer also provides the ability to move nodes using the cursor.

5.2.3. Animation Controls

Functions that manage the transition between different states:

- `updateAnimation()`: A per-frame update that calculates node positions and timers.
- `start[Action]Animation()`: Entry points that initialize the animation state for a specific operation (e.g., `startInsertAnimation`).
- `animTimer / animSpeed`: Variables controlling the progression of the visual transitions.

5.2.4. History & Snapshot Management

Functions enabling debugging and step-by-step visualization:

- `captureSnapshot()`: Captures the current state of all nodes into a `Snapshot` and adds it to the `history` vector.
- `restoreSnapshot()`: Reconstructs the data structure's visual state from a saved `Snapshot`.
- `history`: A sequential list of every state change, allowing the user to scrub forward and backward through the algorithm's execution.

5.3. Main flow example: AVL Tree Insertion

1. **Input:** User types a number into the text box and clicks the `Insert` button.
2. **Initialization:** `startInsertAnimation(value)` is called. The algorithm determines the target insertion path and changes `animMode` to `1` (Animating).
3. **Update Loop:** In the application's `while(!WindowShouldClose())` loop, `updateAnimation()` is triggered every frame. Using a delta timer (`animTimer`), it progresses the current node pointer down the tree, highlighting the respective `activeLine` in the pseudocode array.
4. **Snapshot:** Once the timer threshold is met, `captureSnapshot()` is called, and the tree links the new node, triggering a balance check. If unbalanced, the `leftRotate()` or `rightRotate()` function is applied, and the nodes' `vX/vY` coordinates are updated to slide smoothly across the screen to their new locations.

6. Technical Problems and Solutions

6.1. Backward logic

- **Issue:** Allowing the user to step "backward" in algorithms like Heap pops or Tree deletions is difficult because data is permanently lost or heavily rearranged.
- **Solution:** Store a deep copy of the state for each step of the visualization. I created a `Snapshot` struct that records a deep copy of arrays, tree nodes, and active pseudocode lines at the exact moment before a step executes. Stepping backward simply replaces the current program state with the popped `Snapshot`.

6.2. Graph node overlap

- **Issue:** Randomly generating graph nodes resulted in overlapping vertices and unreadable edge lines.
- **Solution:** Implemented an algorithmic physics simulation (Force-directed Layout, implemented in function `calculateNodePositions()`). This is a graph layout algorithm based on physical simulation that determines node positions by simulating attraction and repulsion forces between nodes. Nodes act like magnets pushing each other away, while edges act as springs pulling them together. Running this simulation for 200 iterations before drawing visually untangles the graph perfectly. I also added the feature that allows users to freely move nodes within the UI using the cursor.

6.3. Compiler conflicts with Windows GUI

- **Issue:** Using `tinyfiledialogs` to open Windows file explorers caused severe compilation crashes because `<windows.h>` and `raylib.h` both define conflicting `Rectangle` structures.
- **Solution:** I instructed the GNU Make build system to compile `tinyfiledialogs.c` separately as an isolated object file, bypassing the namespace collisions entirely while keeping the GUI functioning perfectly.

7. Features Demonstration

The showcase of all the features of the project can be accessed via this link: https://youtu.be/zimY2A1wQLM.

8. Conclusion

The DSA Visualizer successfully meets its goal of providing an educational, stable, and visually engaging sandbox for computer science concepts. It securely integrates an immediate-mode GUI with complex algorithm states without sacrificing performance.

Potential areas for improvement include adding zooming and panning capabilities via a 2D Camera to support much larger datasets, implementing Red-Black Trees or Hash Maps, and allowing users to save/export their drawn graph structures back into text files.

9. References

1. **raylib:** A simple and easy-to-use library to enjoy videogames programming. https://www.raylib.com/
2. **raygui:** Immediate-mode GUI library for Raylib. https://github.com/raysan5/raygui

3. **tinyfiledialogs:** Cross-platform standard dialogs for C/C++. https://sourceforge.net/projects/tinyfiledialogs/
4. **Force-directed Layout:** A graph layout algorithm based on physical simulation that determines node positions by simulating attraction and repulsion forces between nodes. https://g6.antv.antgroup.com/en/manual/layout/force-layout